

1	Azure OpenAI Native Client - Class Design Diagram	1
1.1	Class Design Overview	1
1.1.1	Core Classes	1
1.1.2	External Dependencies	3
1.2	Key Differences from LangChain	3
1.2.1	Removed:	3
1.2.2	Added:	3
1.2.3	Benefits:	4
1.3	Data Flow	4
1.4	Authentication	4
1.4.1	Production (Managed Identity):	4
1.4.2	Development (Azure CLI):	4
1.5	Configuration Example	5

[illegible]

1.1 Class Design Overview

1.1.1 Core Classes

Configuration constants:

- AZURE_OPENAI_ENDPOINT: Azure OpenAI service endpoint
- AZURE_OPENAI_DEPLOYMENT: Model deployment name
- AZURE_OPENAI_API_VERSION: API version (2024-12-01-preview)
- AZURE_OPENAI_SCOPE: OAuth scope for authentication
- MANAGED_IDENTITY_CLIENT_ID: Client ID for managed identity

1.1.1.2 2. AzureOpenAINativeClient (Native Client Manager) File: src/agents/azure-openai-native

Purpose: Manages native OpenAI client lifecycle without LangChain

Key Methods: - `getClientInstance()`: Returns cached or creates new OpenAI client - `selectCredential()`: Chooses ManagedIdentity (prod) or AzureCli (dev) - `createBearerTokenProvider()`: Converts Azure credential to bearer token - `clearCache()`: Clears cache for testing - `initializeClient()`: Creates new OpenAI client

Process: 1. Check cache 2. Select credential based on NODE_ENV 3. Create bearer token provider 4. Initialize native AzureOpenAI client 5. Cache and return

1.1.1.3 3. NativeStreamAdapter (Stream Handler) File: src/agents/azure-openai-stream-adapter

Purpose: Adapts native OpenAI streaming to Pi format

Key Methods: - `streamAzureOpenAINative()`: Main streaming function - `convertContextToMessages()`: Pi → Native OpenAI format - `handleStreamEvents()`: Process streaming chunks - `emitEvents()`: Emit Pi events - `handleErrors()`: Error handling

Process: 1. Create event stream 2. Get native client 3. Convert Pi context to native messages 4. Stream from `chat.completions.create()` 5. Emit Pi-compatible events 6. Handle errors

1.1.1.4 4. PiEmbeddedRunner (Detection & Routing) File: pi-embedded-runner/run/attempt.ts

Purpose: Detects managed identity and routes to native streaming

Detection Logic:

```
normalizedProvider = normalizeProviderId(provider)
providerConfig = config.models.providers[provider]
usesAzureManagedIdentity =
  normalizedProvider === 'azureopenai' &&
  providerConfig.auth === 'managedidentity'

if (usesAzureManagedIdentity) {
  activeSession.agent.streamFn = streamAzureOpenAINative
} else {
  activeSession.agent.streamFn = streamSimple
}
```

1.1.1.5 5. AuthBypassLogic (API Key Bypass) File: pi-embedded-runner/run.ts & compact.ts

Purpose: Bypasses API key requirement for managed identity

Key Logic:

```

if (!apiKeyInfo.apiKey) {
  if (mode !== 'aws-sdk' && mode !== 'managedidentity') {
    throw Error('No API key')
  }

  if (mode === 'managedidentity') {
    authStorage.setRuntimeApiKey(
      provider,
      'MANAGED_IDENTITY_PLACEHOLDER'
    )
  }
}

```

1.1.1.6 6. ConfigurationTypes (Type Definitions) Files: Various type definition files

Key Types: - AuthProfileConfig: { mode, provider } - ModelProviderAuthMode: 'managedidentity' | 'api-key' | 'aws-sdk' | 'oauth' | 'token' - ProviderConfig: { auth, baseUrl, api }

1.1.2 External Dependencies

1.1.2.1 7. AzureIdentity (@azure/identity)

- ManagedIdentityCredential(clientId): Production auth (no keys!)
- AzureCliCredential(): Development auth (az login)
- getBearerTokenProvider(credential, scope): Token provider

1.1.2.2 8. OpenAIClient (@azure/openai)

- AzureOpenAI(config): Native OpenAI client
 - chat.completions.create(): Chat API
 - Native streaming support
 - Direct API integration
-

1.2 Key Differences from LangChain

1.2.1 Removed:

- @langchain/openai dependency
- AzureChatOpenAI wrapper
- LangChain message types
- Intermediate conversion layers

1.2.2 Added:

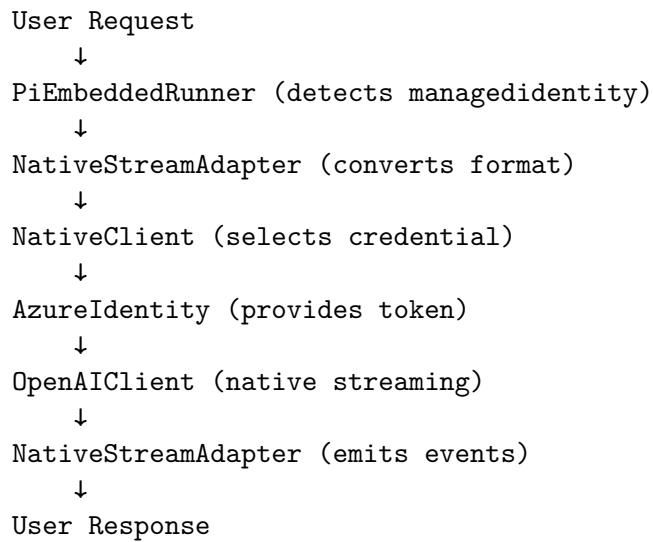
- Direct @azure/openai SDK usage
- Native OpenAI message format

- Simplified architecture

1.2.3 Benefits:

- Fewer dependencies
 - Better performance
 - Simpler code
 - Smaller bundle size
-

1.3 Data Flow



1.4 Authentication

1.4.1 Production (Managed Identity):

1. Detect `NODE_ENV=production`
2. Create `ManagedIdentityCredential(clientId)`
3. Call `getBearerTokenProvider(credential, scope)`
4. Initialize `AzureOpenAI`
5. No API keys needed!

1.4.2 Development (Azure CLI):

1. Detect `NODE_ENV=development`
 2. Create `AzureCliCredential()`
 3. Use tokens from `az login`
 4. Call `getBearerTokenProvider(credential, scope)`
 5. Initialize `AzureOpenAI`
-

1.5 Configuration Example

```
{
  "models": {
    "providers": {
      "azureopenai": {
        "auth": "managedidentity",
        "baseUrl": "https://your-instance.openai.azure.com",
        "api": "2024-12-01-preview"
      }
    }
  }
}
```